

Petal — Project Brief

Petal / Noctworks

June 16, 2026

CONTENTS

title: "Petal — Project Brief" subtitle: "AI-native tax practice management. Frontend is built; this is the backend you're building." author: "Petal / Noctworks"

1. What Petal is	
2. The mental model (domain)	
3. Backend stack — options to decide (nothing is locked yet)	
What we're actually choosing between	
Option A — Convex (reactive TypeScript BaaS)	
Option B — FastAPI + Postgres (Python)	
Option C — Node / TypeScript API + Postgres (NestJS, or tRPC/Hono + Prisma/Drizzle)	
Option D — Supabase (Postgres + Auth + Storage + Realtime, RLS-first)	
Option E — Hybrid: TS app-API + Python AI service (split by concern)	
How to choose (what to weigh when we sit down)	
4. The AI layer (this is the product — get it right)	
4.1 Quarantine — AI output never touches production directly	
4.2 Trust tiers — what Petal may do is set per skill	
4.3 Provenance — every artifact is traceable	
5. Surfaces → what each needs from the backend	
6. The data layer Petal reads from (integrations)	
7. Portal → dashboard flow (the one cross-app handoff)	
8. Security & compliance (we sell trust)	
9. Suggested build order	
10. Where to look in the repo	

title: “Petal — Project Brief” subtitle: “AI-native tax practice management. Frontend is built; this is the backend you’re building.” author: “Petal / Noctworks” date: “June 2026”

1. What Petal is

Petal is an **AI-native practice-management platform for tax firms**. The first customer is a solo Enrolled Agent (Antonio Vazquez, "Vazant EA") with ~200–250 clients. The pitch in one line: *Petal prepares everything; the human approves*.

It is not "tax software with a chatbot bolted on." The AI ("Petal") is the operating layer. It drafts returns, chases documents, answers IRS notices, reconciles books, and surfaces exactly what needs a human decision — and **nothing it produces touches a client record until a human approves it**.

There are two apps:

- **Preparer dashboard** (`/os`) — the firm-facing app. This is what's built and what this brief is mostly about.
- **Client portal** — a separate client-facing app (intake, document upload, e-sign, payments). Lighter; shares the same backend.

Current state: the entire `/os` frontend is built and high-fidelity, running on in-memory fixtures (`lib/fixtures/*`). **There is no backend**. Every number, list, and chat reply is derived from static fixture data at render time. Your job is to make it real.

2. The mental model (domain)

Tax firms don't think in "users and rows." Learn this vocabulary — the whole UI is built on it:

Concept	What it means	Example
Household	The client / relationship hub. Can be individual, business, or mixed.	"Chen Household"
Person	A human inside a household (taxpayer, spouse, owner, partner, bookkeeper).	"Marcus Chen"
Entity	A thing that <i>files</i> a return.	"Golden Dragon LLC"
Engagement (Return)	One entity × one tax year. The core unit of work.	"1040 · 2025"
Expected doc	A checklist item per engagement (requested / received / needs-review / N/A).	"W-2 — Golden Dragon"
Task	The atomic unit of the review queue — the one thing a human must do.	"Approve CP2000 response"
Skill	A repeatable Petal capability (doc chase, books reconcile, notice response...).	"Notice Response"
Skill run	One execution of a skill, with full provenance (sources → reasoning → output).	run #r-cp2000
Notice	An IRS/state notice attached to a household.	"CP2000 — Rodriguez"
Workpaper	The line-by-line trace from a return figure back to its source document.	
Thread	A client conversation across email / SMS / portal / call.	

A household has many entities and people; each entity has engagements; each engagement has expected docs, tasks, and a workpaper.

Everything is derived from these primitives (see `lib/fixtures/firm.ts` for the exact TypeScript interfaces — treat them as the schema starting point).

3. Backend stack — options to decide (nothing is locked yet)

Frontend (done, fixed): Next.js 15 (App Router), React 19, Tailwind CSS v4, deployed on Vercel.

Backend: undecided. This is the conversation we're about to have. Below are five credible stacks with honest trade-offs. (An earlier doc, [docs/BACKEND_ARCHITECTURE.md](#), sketches a Convex design — treat it as *one prior proposal*, not a decision.) Whatever we pick, the requirements in Sections 4 and 8 hold: auth-scope every read, audit every write, and quarantine every AI output behind human approval.

WHAT WE'RE ACTUALLY CHOOSING BETWEEN

The hard parts of Petal that the stack has to serve well: a **live-updating UI** (approving a task moves counts everywhere), a **real AI/agent layer** (drafting, OCR, tool use, citations), **PII security** (a compliance-sensitive app), and **lots of integrations** (OAuth, webhooks, scheduled syncs). The five options weight those differently.

OPTION A — CONVEX (REACTIVE TYPESCRIPT BAAS)

Reactive DB + serverless functions + auth + file storage + cron, all TypeScript.

- **Pros:** Reactive queries map 1:1 to the live UI (approve a task and every count moves, no cache plumbing); end-to-end TypeScript shared with the frontend; batteries included; fastest path to a working app.
- **Cons:** No row-level security — every function enforces auth/ownership by hand (discipline risk on a PII app); the AI/OCR work still wants Python, so you call out to a separate service anyway; vendor lock-in; smaller ecosystem.
- **Best when:** TS-first team, you want the fastest live demo, and you'll accept app-layer security discipline.

OPTION B — FASTAPI + POSTGRES (PYTHON)

Explicit Python API over Postgres; add a job queue (Celery/Arq) and object storage (S3/GCS).

- **Pros:** Python is the native home of the AI/agent + Document-AI ecosystem — Petal's crown jewel; maximum control and transparency; Postgres gives real row-level security; huge ecosystem and hiring pool; excellent for integrations/webhooks.
- **Cons:** Not reactive out of the box (live counts need websockets/SSE or polling); different language from the TS frontend (no shared types without generating an OpenAPI client); more pieces to assemble and run (auth, storage, jobs).
- **Best when:** the AI layer is the differentiator and you want Python at the core with explicit control.

OPTION C — NODE / TYPESCRIPT API + POSTGRES (NESTJS, OR TRPC/HONO + PRISMA/DRIZZLE)

A typed TS backend over Postgres, sharing types with the frontend.

- **Pros:** One language across the whole stack; tRPC gives end-to-end type-safety frontend↔backend (fewer bugs, fast iteration); Postgres RLS available; modern ecosystem; you own the architecture.
- **Cons:** AI/agent ecosystem is thinner in TS than Python (fine for "call the model API," weaker for OCR/ML — may still want a Python sidecar); you assemble reactivity yourself; more boilerplate than a BaaS.
- **Best when:** you value one-language + shared-types velocity and your AI work is mostly orchestrating model APIs.

OPTION D — SUPABASE (POSTGRES + AUTH + STORAGE + REALTIME, RLS-FIRST)

Managed Postgres with real row-level security, plus auth/storage/realtime — but it's just Postgres underneath, so portable.

- **Pros:** Security enforced *at the database* via Postgres RLS — the strongest default for a PII/compliance app; batteries included (auth, storage, realtime); fast to build; open-source and portable (no hard lock-in); pairs with TS edge functions or a separate Python service.
- **Cons:** Edge functions are too thin for heavy AI orchestration — pair Supabase with a dedicated agent service; realtime is row-level, less elegant than query-level reactivity; some platform-specific patterns.
- **Best when:** you want database-enforced security and managed infra, and will run the agent loop as a separate service.

OPTION E — HYBRID: TS APP-API + PYTHON AI SERVICE (SPLIT BY CONCERN)

A TS backend (or BaaS) for app/CRUD/realtime, plus a dedicated Python service for the agent loop, OCR, and model orchestration.

- **Pros:** Right tool per job — TS for the reactive app + shared types, Python for the AI; each half stays simple and testable; best-of-both.
- **Cons:** Two services to build, deploy, and monitor; the contract between them is now your problem; more ops overhead early.
- **Best when:** you expect the AI layer to get complex and want it isolated, and can absorb the extra ops.

HOW TO CHOOSE (WHAT TO WEIGH WHEN WE SIT DOWN)

Criterion	Pulls toward
Team familiarity / hiring	whatever the team ships fastest in
Live-updating UI matters a lot	Convex (A); else build realtime (B/C/D)
AI/agent + OCR is the hard part	Python at the core (B), or hybrid (E)
Security model preference	DB-enforced RLS (B/C/D) vs app-layer discipline (A)
One language + shared types	Convex (A) or TS+Postgres (C)
Integrations, webhooks, cron	all viable; Python/Postgres are most batteries-flexible
Speed to first demo vs long-term control	BaaS (A/D) for speed; B/C/E for control
Lock-in / portability	Postgres-based (B/C/D) is most portable

My read, to argue about, not to adopt: the live-reactive UI says “Convex,” the AI layer says “Python,” and the PII compliance story says “Postgres RLS.” That tension is exactly why **Option E (hybrid)** or **Option D (Supabase + a Python agent service)** are worth serious weight — but the right answer depends on team and timeline, which is your call.

4. The AI layer (this is the product — get it right)

Petal's AI is not a feature; it's the spine. Three rules govern it:

4.1 QUARANTINE — AI OUTPUT NEVER TOUCHES PRODUCTION DIRECTLY

Every AI result lands in a **quarantine table** with `status: "pending_review"`. A human approves before it is promoted to the real record. The audit log records who approved what, when. This is the safety guarantee we sell to CPAs and it is **non-negotiable**.

4.2 TRUST TIERS — WHAT PETAL MAY DO IS SET PER SKILL

Each skill has a **trust tier** that gates its write power:

- **Suggest** — drafts only; human does everything.
- **Draft & queue** — Petal prepares and queues a task; human approves each.
- **Act with approval** → **Act autonomously** — graduated autonomy the firm dials up per skill over time (e.g., "Doc Chase has been right 40 times → promote to autonomous, acts after 24h unless stopped").

The API/MCP layer enforces these tiers too: reads are open to authorized keys; **every write is gated by the relevant skill's trust tier**. Returns are *drafts only* — *Petal never files*.

4.3 PROVENANCE — EVERY ARTIFACT IS TRACEABLE

Every skill run stores its **sources** → **reasoning** → **output**. The UI exposes "View run" everywhere; a workpaper line traces back to the exact source document and page. The backend must persist this chain for every run.

Model strategy (from the architecture doc): Google Document AI for OCR; a small model (GPT-4o-mini class) for drafts; a strong model (GPT-4o class) for compliance-sensitive work. *Default to the latest, most capable models when we build the agent loop*. All output quarantined until approved.

5. Surfaces → what each needs from the backend

The `/os` app's routes map directly to backend domains. For each, the backend must serve **reactive queries** (live counts) and **mutations** (the actions), all auth-scoped.

- **Home / Today** — the morning brief: what needs you, filing readiness, the weekly digest. Reads across tasks, engagements, notices, threads.
 - **Tasks** — the review queue. The most important surface. Group by status; each task has exactly one primary verb (Approve / Decide / Nudge...). Approving resolves the task and moves every dependent count. Needs: task CRUD, status transitions, the approve→promote-from-quarantine flow, assignment.
 - **Clients / Returns / People** — the records. Household / entity / engagement detail with documents, tasks, messages, billing, positions, compliance. Heavy read surface; assignment + notes are writes.
 - **Documents** — per-engagement checklists + a firm-wide file store. Needs: file storage (Convex storage), upload, OCR pipeline → expected-doc matching, status (requested/received/needs-review).
 - **Inbox / Messages** — threads across email/SMS/portal/call. Needs: inbound capture (email/SMS webhooks), outbound send (gated), thread status.
 - **Notices** — IRS/state notices, each with a response workflow (Petal drafts; human approves before it mails).
 - **Billing** — invoices, deposits, collected/balance, payment method. Stripe integration.
 - **Calendar** — events + pre-call briefs (reads Google Calendar).
 - **Ask Petal (chat)** — the agentic chat on `/os/ask` and in every record rail. Needs: an agent loop that can read firm data (scoped), call skills, attach documents, stream answers, and cite sources. Today it's a scripted demo bank (`lib/fixtures/demo-chat.ts`) — replace it with a real retrieval + tool-use agent.
 - **Skills / Knowledge / Activity** — the skill library + runs, the firm "Constitution" (reference knowledge for grounding), and the immutable activity log (the flight recorder — every run and approval, append-only).
 - **Settings** — firm profile, members & roles/permissions (editable matrix: owner/reviewer/preparer/admin × sign_returns, efile, approve_drafts, prepare_returns, manage_billing, manage_team, intake_docs), trust dials, integrations, and the **API & MCP** developer surface.
-

6. The data layer Petal reads from (integrations)

Petal grounds its work in the firm's real stack. These are the connectors to build (OAuth, scoped, with writes gated by trust tier):

- **Accounting:** QuickBooks Online, Xero
- **Payroll:** Gusto
- **Tax & e-file:** OLT/Drake-class transmitter, IRS e-Services (transcripts, CAF/8821)
- **Docs & e-sign:** Google Drive, DocuSign (8879s)
- **Email & calendar:** Gmail, Google Calendar
- **Payments:** Stripe (prep fees, deposits)
- **POS / client data:** Square
- **Comms:** Slack (alerts)

Two consumption paths matter equally: the **app** uses these; and Petal exposes the firm's OS over **REST + a native MCP server** (OAuth 2.1 + PKCE) so Claude/Cursor/Zapier can read it. See `lib/os-api.ts` for the intended resource scopes and write policies.

7. Portal → dashboard flow (the one cross-app handoff)

When a client finishes portal intake, the backend must, atomically:

1. Create/Update the client record with `intakeAnswers`.
2. Auto-generate the document checklist from those answers.
3. Create the appointment from the chosen slot.
4. Create the deposit payment record (Stripe).
5. Post a system message + create an action item for the preparer.

Client messages flow into the preparer's Inbox; automated status/doc/payment replies use `senderType: "system"`.

8. Security & compliance (we sell trust)

- **PII encrypted at rest** (AES-256). Payment data never touches our servers (Stripe/Square hold PCI scope).
 - **OTP** for client portal access.
 - **Audit everything** — append-only, on every client-data mutation.
 - **§7216 consent** templates for disclosure; **WISP** (written info security plan) on file.
 - **Zero-data-retention** agreements with model providers; firm data is never used to train models and never shared across firms.
 - The AI-quarantine pattern (Section 4.1) is itself a compliance control.
-

9. Suggested build order

A pragmatic sequence, stack-agnostic — each phase is demoable:

1. **Foundations** — project + auth (preparer + client identities), the auth-scope middleware, the append-only audit table, and the core schema (households → entities → engagements → docs → tasks). Port the `lib/fixtures/firm.ts` interfaces to real tables.
2. **Read-through** — wire the `/os` read surfaces to live queries (Today, Tasks, Clients, Returns). Kill the fixtures one surface at a time behind a flag.
3. **The queue** — task mutations + the quarantine→approve→promote flow + assignment. This is the heartbeat of the app.
4. **Documents** — file storage, upload, OCR pipeline (Document AI), expected-doc matching.
5. **Agent loop** — replace the scripted chat with a real retrieval + tool-use agent (read scoped firm data, call skills, cite sources, attach docs). Trust-tier enforcement on every tool that writes.
6. **Integrations** — QBO/Xero, Gmail, Stripe, IRS e-Services first (highest-leverage), then the rest. OAuth + webhooks + sync jobs.
7. **API & MCP** — the public REST + MCP server with OAuth 2.1/PKCE and tier-gated writes.

Infra cost lands roughly **\$80–150/month** at this scale regardless of stack (host + database + model usage + Stripe pass-through); the model/OCR usage dominates, not the framework.

10. Where to look in the repo

- `lib/fixtures/firm.ts` — **the data model**. Every interface (Household, Entity, Engagement, ExpectedDoc, Task, Skill, SkillRun, Notice, Workpaper, Thread, ...). Start here.
- `lib/fixtures/derive.ts` — how every UI number is computed from primitives. This is effectively the **query spec** — your Convex queries should reproduce these derivations.
- `lib/os-api.ts` — intended REST/MCP resources, scopes, and write policies.
- `app/os/*` — the surfaces; each folder is a domain.
- `docs/BACKEND_ARCHITECTURE.md` — a *prior proposal* (assumes Convex). The quarantine pattern, portal flow, and AI provider strategy in it are reusable; the Convex choice is **not** decided.
- `docs/PETAL_OS_SPEC.md`, `docs/PRODUCT_BIBLE.md` — product depth.

The golden rule, restated: auth-scope every read, audit every write, and **nothing AI-generated reaches a real record without a human approval**. Build around that and everything else follows.